



JOHNS HOPKINS
UNIVERSITY

Emily Riehl

A new paradigm for mathematical proof?

Mathematical Proof in the Age of AI:
Implications of the Use of Artificial Intelligence in Mathematical Research

Plan



1. A new paradigm for human-generated mathematics?
2. A new paradigm for AI-generated mathematics?
3. Predications about the future



1

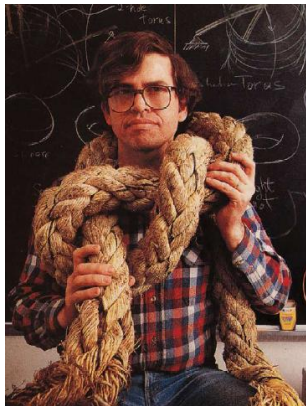
A new paradigm for human-generated
mathematics?

What is mathematics?



Mathematicians tend to take a more expansive view of mathematics than the general public, essentially because we subscribe to the following recursive answer to the question:

“What is mathematics?”



“Mathematics is the smallest subject satisfying the following:

- Mathematics includes the natural numbers and plane and solid geometry.
- Mathematics is that which mathematicians study.
- Mathematicians are those humans who advance human understanding of mathematics.”

— William P. Thurston,
“On proof and progress in mathematics”

A theorem about the natural numbers



In 1637, **Pierre de Fermat** claimed in a marginal note that he had a marvelous proof to the following statement in but provide no details:

Fermat's Last Theorem The equation $x^n + y^n = z^n$ has no integer solutions with $xyz \neq 0$ and $n \geq 3$.

In 1993, **Andrew Wiles** announced a proof, a modern tour-de-force using elliptic curves and modular forms.



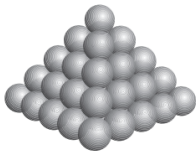
His proof contained a small mistake that took over a year to correct, with help from **Richard Taylor**. Their work was published in 1995 in the *Annals of Mathematics* in two papers totaling 129 pages.

A theorem about solid geometry

In 1998, Thomas Hales announced a proof of a 1611 conjecture of Johannes Kepler, via a “**proof by exhaustion**” involving the checking of many individual cases using a computer to solve linear programming problems. After four years, a panel of 12 referees reported they were 99% certain that the proof was correct, but could not check all the computer calculations.

THEOREM 1.1 (The Kepler conjecture). *No packing of congruent balls in Euclidean three space has density greater than that of the face-centered cubic packing.*

This density is $\pi/\sqrt{18} \approx 0.74$.



The unabridged version of the paper, which was published in 2005 in the *Annals of Mathematics*, came to 339 pages, with around 3 gigabytes of computer artifacts.

Proof as peer review



TL;dr: Rigorous proofs of simple sounding theorems can be extremely complicated.

Why has mathematics largely avoided the replication crisis
that has confronted other fields?

Peer review in theory: careful refereeing should lead to an error-free publications.

Unfortunately, the mathematical literature contains famous mistakes as well as
contradictory theorems.

Peer review in practice: in theory any proof should be **reproducible** by any reader —
allowing the reader to understand for themselves why the result is true.

When papers have enough readers, mistakes are eventually caught.

Importantly: human mathematicians are careful in claiming they have a proof!

Proof as peer review?



But sometimes these ideals break down:

One Fields medalist was dismayed to find mistakes in his published, well-studied papers:

“A technical argument by a trusted author, which is hard to check and looks similar to arguments known to be correct, is hardly ever checked in detail.”

— Vladimir Voevodsky

Another Fields medalist expressed doubts about a particular proof he had discovered — and also doubted that anyone else would check it:

“...while I was very happy to see many study groups on condensed mathematics throughout the world, to my knowledge all of them have stopped short of this proof. (Yes, this proof is not much fun...)”

— Peter Scholze

In some papers, too much work is “left to the reader”

and not enough readers are up to the task!

A new paradigm for mathematical proof?



THE EQUIVARIANT MODEL STRUCTURE ON CARTESIAN CUBICAL SETS

STEVE AWODEY, EVAN CAVALLO, THIERRY COQUAND, EMILY RIEHL, AND CHRISTIAN SATTLER

ABSTRACT. We develop a constructive model of homotopy type theory in a Quillen model category that classically presents the usual homotopy theory of spaces. Our model is based on presheaves over the cartesian cube category, a well-behaved Eilenberg-Zilber category. The key innovation is an additional equivariance condition in the specification of the cubical Kan fibrations, which can be described as the pullback of an interval-based class of uniform fibrations in the category of symmetric sequences of cubical sets. The main technical results in the development of our model have been formalized in a computer proof assistant.

CONTENTS

1. Introduction	2
1.1. Interpreting homotopy type theory	2
1.2. Cubical interpretations	3
1.3. Cubical model structures	3
1.4. Standard homotopy theory	4
1.5. The equivariant cubical model	4
1.6. Results	7
1.7. Related and future work	10
1.8. Acknowledgments	11
2. Notions of fibred structure, universes, and realignment	12
2.1. Locally representable and relatively acyclic notions of fibred structure	12
2.2. Monomorphisms and uniform trivial fibrations	18
2.3. Universes and realignment	23
3. Cylindrical model structures	25
3.1. Cylindrical premodel structures	26
3.2. Brown factorizations	28
3.3. Equivalence extension property	30
3.4. The Frobenius condition	32
3.5. Univalence	35
3.6. Fibrant universes	37
3.7. Fibration extension property and 2-of-3	40
4. The interval model structure on cubical species	41
4.1. Groupoid-indexed diagram categories	41
4.2. Cubical species and the symmetric interval	42
4.3. The cylindrical premodel structure on cubical species	44
4.4. The cubical species model of homotopy type theory	51
5. The equivariant model structure on cubical sets	52
5.1. From cubical species to equivariant cubical sets	53
5.2. The cylindrical premodel structure on cubical sets	54
5.3. The equivariant cubical sets model of homotopy type theory	58

6. The equivalence with classical homotopy theory	60
6.1. Triangulation	61
6.2. Eilenberg-Zilber categories	68
6.3. The equivariant model structure is the test model structure	74
Appendix A. Type-theoretic development and formalization	75
A.1. Introduction	75
A.2. Judgments of the homotopical interpretation	76
A.3. Cubes and cofibrations	77
A.4. Partial elements and contractible types	77
A.5. Filling and equivariant filling	78
A.6. The Frobenius condition	79
A.7. Other type formers	80
A.8. Tiny interval and universes	80
References	83

Software programs called **computer proof assistants** can certify the correctness of a mathematical proof that has been written in a precise formal language.

- Traditionally such proofs are laboriously encoded by human mathematicians (**formalization**).
- Generative AI could be trained to output proof text in a format that could be checked by a computer proof assistant (**autoformalization**).

Computer proof verification



Our 87 page preprint is accompanied by a library of formalized proofs checked by the computer proof assistant **Agda**:

The main definition takes just a few lines to encode ↴

```
{-
```

```
-----  
Formalization of an equivariant cartesian cubical set model of type theory  
-----
```

This formalization accompanies the article

The equivariant model structure on cartesian cubical sets.
Steve Awodey, Evan Cavallo, Thierry Coquand, Emily Riehl, & Christian Sattler.
<https://arxiv.org/abs/2406.18497>

The contents of the formalization are outlined in Appendix A of the article.

The formalization defines a model of homotopy type theory inside an extensional type theory augmented with a flat modality and axioms postulating *shapes* (among them an *interval*) and a cofibration classifier. The results can in particular be externalized in the category of cartesian cubical sets.

The code has been tested with Agda version 2.6.4.
The source is available at

github.com/ecavallo/equivariant-cartesian

and there is an HTML interface at

ecavallo.github.io/equivariant-cartesian

For reference (see the file `equivariant.agda-lib` in the source), the formalization is compiled with the flags

```
--with-K  
--cohesion --flat-split  
--no-import-sorts  
--rewriting
```

In particular, the `--with-K` flag enables axiom K (uniqueness of identity proofs), while the `--cohesion` and `--flat-split` flags enable the flat modality (see the module `axiom.flat` for more information).

```
-}
```

```
--+ The equivariance condition on local filling structures associated to a shape  
--+ homomorphism  $\sigma : S \rightarrow T$ . Filling an open box over T and then composing with  $\sigma$  should be  
--+ the same as composing the box with  $\sigma$  and then filling over S.
```

```
LocalEquivariance : {S T : Shape} ( $\sigma : \text{Shape}[S, T]$ ) ( $A : (T) \rightarrow \text{Type } \ell$ )  
   $\rightarrow \text{LocalFillStr } T A \rightarrow \text{LocalFillStr } S (A \circ \langle \sigma \rangle) \rightarrow \text{Type } \ell$   
LocalEquivariance  $\sigma$  liftT liftS =  
   $\forall r \text{ box } s \rightarrow$   
   $\text{reshapeFiller } \sigma (\text{liftT } (\langle \sigma \rangle r) \text{ box}) .\text{fill } s .\text{out}$   
   $= \text{liftS } r (\text{reshapeBox } \sigma \text{ box}) .\text{fill } s .\text{out}$ 
```

```
Equivariance : {S T : Shape} ( $\sigma : \text{Shape}[S, T]$ ) ( $\Gamma : \text{Type } \ell$ ) ( $A : \Gamma \rightarrow \text{Type } \ell'$ )  
   $\rightarrow \text{FillStr } T A \rightarrow \text{FillStr } S A \rightarrow \text{Type } (\ell \cup \ell')$   
Equivariance ( $T = T$ )  $\sigma \{ \Gamma \} A \text{ fillT fillS} =$   
  ( $\gamma : \Gamma \wedge T$ )  $\rightarrow \text{LocalEquivariance } \sigma (\text{fillT } \gamma) (\text{fillS } (\gamma \circ \langle \sigma \rangle))$ 
```

```
--+ Definition of an equivariant fibration structure.
```

```
record FibStr ( $\Gamma : \text{Type } \ell$ ) ( $A : \Gamma \rightarrow \text{Type } \ell'$ ) :  $\text{Type } (\ell \cup \ell')$  where  
  constructor makeFib  
  field
```

```
--+ We have a filling structure for every shape.
```

```
lift : ( $S : \text{Shape}$ )  $\rightarrow \text{FillStr } S A$ 
```

```
--+ The filling structures satisfy the equivariance condition.
```

```
vary :  $\forall S T (\sigma : \text{Shape}[S, T]) \rightarrow \text{Equivariance } \sigma A (\text{lift } T) (\text{lift } S)$ 
```

Nevertheless, as is typical, the period from submission to publication took 18 months.

What are computer proof assistants?



A **computer proof assistant** or **interactive theorem prover** — such as **Agda**, **HOL Light**, **Isabelle**, **Lean**, **Mizar**, or **Rocq** (née **Coq**) — is a computer program that:

- knows the rules of a logical formal system (e.g., a foundation for mathematics), which a trusted core program (the **kernel**) uses to check the correctness of proofs
- is programmed (via the **elaborator**) to interpret statements written in an expressive formal language (the **vernacular**) used to encode definitions, theorems, and proofs.

Formal proofs, written by a human user, are developed interactively with the computer.

- The mathematician inputs each line of their proof in a precise syntax.
- The computer checks that the logical argument supplied by the user produces a valid deduction of the claimed mathematical statement.

Aside: modern proof assistants often use a newer formal system — **dependent type theory** — in place of traditional Zermelo-Fraenkel set theory and first order logic.

A formalized proof



▼ IrrationalDemo.lean ●

FYS270LeanProject > ▼ IrrationalDemo.lean > { } Real > sqrt_two_irrational

```
import Mathlib.Analysis.Real.Sqrt
import Mathlib.Data.Nat.Prime.Defs
import Mathlib.Data.Rat.Sqrt

/-- A proof of the (ir)rationality of the square root of 2. -/

open Nat
You, 5 hours ago | author (You)
namespace Real

/-- The square root of 2 is irrational. That is, there do not exist
natural numbers `m` and `n`, with `gcd m n = 1`, so that
`sqrt 2 = m / n`. -/
theorem sqrt_two_irrational :
  ¬ (∃ (m n : ℕ), gcd m n = 1 ∧ sqrt 2 = m / n) := by
```

Lean InfoView ×

▼ IrrationalDemo.lean:16:2

▼ Tactic state

1 goal

| ⊢ ¬∃ m n, m.gcd n = 1 ∧ √2 = m / n

▼ Messages (0 1)

▼ IrrationalDemo.lean:13:0

Goals accomplished!

► All Messages

▼ IrrationalDemo.lean ●

FYS270LeanProject > ▼ IrrationalDemo.lean > { } Real > sqrt_two_irrational

```
/-- The square root of 2 is irrational. That is, there do not exist
natural numbers `m` and `n`, with `gcd m n = 1`, so that
`sqrt 2 = m / n`. -/
theorem sqrt_two_irrational :
  ¬ (∃ (m n : ℕ), gcd m n = 1 ∧ sqrt 2 = m / n) := by
  -- This is a proof of negation, so we assume that `∃ m n : ℕ`
  ⚡ -- so that `gcd m n = 1` and `sqrt 2 * n = m`.
  intro {m, n, hyp, eq}
```

Lean InfoView ×

▼ IrrationalDemo.lean:17:23

▼ Tactic state

1 goal

m n : ℕ

hyp : m.gcd n = 1

eq : √2 = m / n

| ⊢ False

▼ Messages (0 1)

A formalized proof



▼ IrrationalDemo.lean ●

FYS270LeanProject > ▼ IrrationalDemo.lean > { } Real > sqrt_two_irrational

```
/- The square root of 2 is irrational. That is, there do not exist
natural numbers `m` and `n`, with `gcd m n = 1`, so that
`sqrt 2 = m / n`. -/
theorem sqrt_two_irrational :
  ¬ (∃ (m n : ℕ), gcd m n = 1 ∧ sqrt 2 = m / n) := by
  -- This is a proof of negation, so we assume that `∃ m n : ℕ`
  -- so that `gcd m n = 1` and `sqrt 2 * n = m`.
  intro (m, n, hyp, eq)
  -- Using `eq : sqrt 2 = m / n` we prove that `2 * n ^ 2 = m ^ 2`.
  have eq2 : 2 * n ^ 2 = m ^ 2 := by
    have eq' := congrArg (fun x => x * x) eq
    simp only [ofNat_nonneg, mul_self_sqrt] at eq'
    apply cast_injective (R := ℝ)
    simp only [cast_mul, cast_ofNat]
    rw [eq']
    grind
  -- Using `eq2 : 2 * n ^ 2 = m ^ 2` and the fact that `2` is prime,
  -- we show `2 | m`.
  have lem1 : 2 | m :=
    Prime.dvd_of_dvd_pow (prime_iff.mp (prime_two)) ((n ^ 2, eq2.symm))
```

Lean InfoView ×

▼ IrrationalDemo.lean:30:2

▼ Tactic state

1 goal

m n : ℕ

hyp : m.gcd n = 1

eq : $\sqrt{2} = \frac{m}{n}$

eq2 : $2 * n^2 = m^2$

lem1 : $2 \mid m$

False

▼ Messages (1)

▼ IrrationalDemo.lean:13:0

Goals accomplished!

► All Messages

A formalized proof



▼ IrrationalDemo.lean ●

FYS270LeanProject > ▼ IrrationalDemo.lean > {} Real > sqrt_two_irrational

```
have lem1 : 2 | m :=
  Prime.dvd_of_dvd_pow (prime_iff.mp (prime_two)) ((n ^ 2, eq2.symm))
-- Unpacking the definition of the type `2 | m`, `lem1` actually
-- proves `∃ k : ℕ, m = 2 * k`.
have (k, eq3) : ∃ k : ℕ, m = 2 * k := lem1
-- Substituting `eq3` into `eq2` proves `eq4 : 2 * n ^ 2 = (2 * k) ^ 2`.
have eq4 : 2 * n ^ 2 = (2 * k) ^ 2 := by
  rw [eq2, eq3]
-- By basic algebra, `eq4` simplifies to `eq5`.
have eq5 : n ^ 2 = 2 * k ^ 2 := by grind
-- Using `eq5 : n ^ 2 = 2 * k ^ 2` and the fact that `2` is prime,
-- we show `2 | n`.
have lem3 : 2 | n :=
  Prime.dvd_of_dvd_pow (prime_iff.mp (prime_two)) ((k ^ 2, eq5))
-- Since `2 | m` and `2 | n`, it follows that `2 | gcd m n`.
have lem4 : 2 | gcd m n :=
  Nat.dvd_gcd lem1 lem3
-- Since `2 | gcd m n` we have `gcd m n ≠ 1`.
have lem5 : gcd m n ≠ 1 :=
  Nat.not_coprime_of_dvd_of_dvd Nat.one_lt_two lem1 lem3
-- We now have `hyp : gcd m n = 1` and `lem5 : gcd m n ≠ 1`.
-- This is our contradiction!
-- Now `absurd hyp lem5` can be used to give an element of any type.
exact absurd hyp lem5
```

Lean InfoView ×

▼ IrrationalDemo.lean:51:2

▼ Tactic state

1 goal

```
m n : ℕ
hyp : m.gcd n = 1
eq : √2 = ↑m / ↑n
eq2 : 2 * n ^ 2 = m ^ 2
lem1 : 2 | m
k : ℕ
eq3 : m = 2 * k
eq4 : 2 * n ^ 2 = (2 * k) ^ 2
eq5 : n ^ 2 = 2 * k ^ 2
lem3 : 2 | n
lem4 : 2 | m.gcd n
lem5 : m.gcd n ≠ 1
├ False
```

▼ Messages (1)

▼ IrrationalDemo.lean:13:0

Goals accomplished!

► All Messages

A library-quality formalized proof



In **Lean**'s mathematics library, the proof is more abstract:

```
@[to_additive]
lemma Irreducible.not_isSquare (ha : Irreducible x) : ¬IsSquare x := by
  rw [isSquare_iff_exists_sq]
  rintro ⟨y, rfl⟩
  exact not_irreducible_pow (by decide) ha

theorem irrational_sqrt_natCast_iff {n : ℕ} : Irrational (√n) ↔ ¬IsSquare
n := by
  rw [← Rat.isSquare_natCast_iff, ← irrational_sqrt_ratCast_iff_of_nonneg
    n.cast_nonneg,
    Rat.cast_natCast]

theorem Nat.Prime.irrational_sqrt {p : ℕ} (hp : Nat.Prime p) : Irrational
(√p) :=
  irrational_sqrt_natCast_iff.mpr hp.not_isSquare

/-- **Irrationality of the Square Root of 2** -/
theorem irrational_sqrt_two : Irrational (√2) := by
  simpa using Nat.prime_two.irrational_sqrt
```

▼Tactic state

No goals

► All Messages (⊗ 3)

The irrationality of 2 is a special case of the irrationality for any prime, which is a deduced from the irrationality of the square root of any non-square number, and primes are not squares by a special case of a result about irreducible elements of a monoid.

A new paradigm for proof writing?



Computer formalization is a not yet widely practiced method of interactively developing and communicating rigorous mathematical proofs using a computer proof assistant. The formalization must either:

- include formalizations of all prerequisite definitions and theorems statements¹
- or may refer to a library of formalized mathematics.

To a human user of an interactive theorem prover, writing a formal proof feels like writing code in a programming language, but with useful real-time feedback:

- typos or conceptual mistakes may be pointed out by “type-checking errors”
- the proof assistant often communicates the standing assumptions and yet-to-be proven objectives midway through a complex proof.

Warning: the proof assistant cannot check whether the formalized definitions or theorem statements accurately capture the mathematical ideas intended by the user!

¹It is often possible to “assume” or “admit” some theorems without proof — “sorry” — but those theorems must nevertheless be stated formally.

Warning: a formalized proof?!?

Lean accepts the following proof that the square root of 2 is rational!



```

IrrationalDemo.lean
FYS270LeanProject > IrrationalDemo.lean > { } Rat

namespace Rat

/-- The square root of 2 is rational. That is, there exist natural
numbers `m` and `n`, with `gcd m n = 1`, so that `sqrt 2 = m / n`. -/
theorem sqrt_two_rational :
  ∃ (m n : ℕ), gcd m n = 1 ∧ sqrt 2 = m / n := by
  use 1
  use 1
  constructor
  · exact rfl
  · rw [Rat.sqrt_ofNat, Nat.sqrt_two]
    simp only [Nat.cast_one, ne_eq, one_ne_zero, not_false_eq_true,
div_self]
  You, yesterday • complete version of the irrational formalizatio...
end Rat
```

Lean InfoView

▼ IrrationalDemo.lean:66:0

▼ Tactic state

No goals

► All Messages

```

IrrationalDemo.lean
FYS270LeanProject > IrrationalDemo.lean > { } Real

namespace Real

/-- The square root of 2 is irrational. That is, there do not exist
natural numbers `m` and `n`, with `gcd m n = 1`, so that
`sqrt 2 = m / n`. -/
theorem sqrt_two_irrational :
  ¬ (∃ (m n : ℕ), gcd m n = 1 ∧ sqrt 2 = m / n) := by
```

Lean InfoView

▼ IrrationalDemo.lean:52:0

▼ Tactic state

No goals

► All Messages

A new paradigm for proof checking?

“A technical argument by a trusted author, which is hard to check and looks similar to arguments known to be correct, is hardly ever checked in detail.”

— Vladimir Voevodsky

“...while I was very happy to see many study groups on condensed mathematics throughout the world, to my knowledge all of them have stopped short of this proof. (Yes, this proof is not much fun...)”

— Peter Scholze

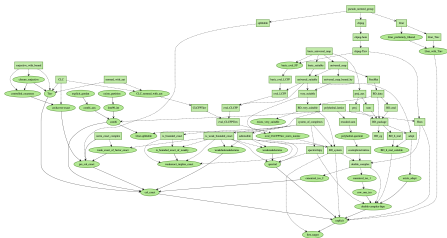
Voevodsky and Scholze both turned to computer formalization to resolve their doubts about the veracity of their own proofs.

MATHEMATICS

The Origins and Motivations of Univalent Foundations

*A Personal Mission to Develop Computer Proof
Verification to Avoid Mathematical Mistakes*

By Vladimir Voevodsky • Published 2014



Large scale computer-verified proofs



When human referees failed to fully certify his proof of the Kepler conjecture, Hales launched a project to verify the result himself in a computer proof assistant. Eleven years later, the full proof was formally verified in the proof assistants **HOL Light** and **Isabelle**. The formalization was described in an accompanying 29 page paper with 22 authors.

A FORMAL PROOF OF THE KEPLER CONJECTURE

THOMAS HALES¹, MARK ADAMS^{2,3}, GERTRUD BAUER⁴,
TAT DAT DANG⁵, JOHN HARRISON⁶, LE TRUONG HOANG⁷,
CEZARY KALISZYK⁸, VICTOR MAGRON⁹, SEAN MC LAUGHLIN¹⁰,
TAT THANG NGUYEN⁷, QUANG TRUONG NGUYEN¹,
TOBIAS NIPKOW¹¹, STEVEN OBUA¹², JOSEPH PLESO¹³, JASON RUTE¹⁴,
ALEXEY SOLOVYEV¹⁵, THI HOAI AN TA⁷, NAM TRUNG TRAN⁷,
THI DIEP TRIEU¹⁶, JOSEF URBAN¹⁷, KY VU¹⁸ and
ROLAND ZUMKELLER¹⁹



In September 2024, Kevin Buzzard launched a project to verify a modern proof of Fermat's last theorem in the computer proof assistant **Lean**, motivated by the goals of:

- improving human understanding of the proof and
- extending **Lean**'s mathematics library to include deep results from modern number theory.

Moral: proofs at the frontier of mathematics are formalizable, but only with monumental human effort via large-scale collaborations ... or perhaps with sufficiently powerful AI?



2

A new paradigm for AI-generated
mathematics?

August 1, 2026



August 1, 2026 Publication

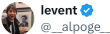
Ten advances in mathematics and theoretical computer science

[Read the paper ↗](#) [Read the reasoning walkthroughs ↗](#)

The 10 advances, including results in discrete geometry, algebra, graph theory, and complexity theory, were presented in a 253 page paper, with each result accompanied by a formalization in Lean.

We provide new results for the following problems. The results were achieved by an internal version of Astra, our next major model. The total number of tokens needed to find solutions to these problems would cost roughly \$2,000 at Sol API rates. These arguments were then prepared into manuscripts by humans with the same model. Afterward, the model formalized each argument in a Lean certificate. We are also releasing for each solution a model's narration of its thinking process.

August 23, 2026



Please welcome to the world a beautiful new geometric object, to do with a problem i've always loved. claude really contains multitudes:D Does S^6 admit a complex structure?

Yup

5:31 PM · Aug 23, 2026 · 1.3M Views

The literature, dating back to 1947, contains several mistaken disproofs. Boris Alexeev from OpenAI produced a Lean formalization four days later.

Formalization of the solution to the Hopf problem

The six-sphere admits a complex manifold structure compatible with its standard topology.

Based on [A compact complex threefold fibred by tori over the projective line, and the six-sphere](#), originally shared on X by Levent Alpoglu.

The repository includes a Comparator setup, with the statement adapted from the [Formal Conjectures project](#).

```
lake update
lake exe cache get
lake build lean4export
lake exe comparator comparator/config.json
```

[Type-check it online!](#)



levent @__alpage__ · Aug 23

i always like to have short certificates that people can use to verify in full with their expertise and/or with one of today's super amazing models. this one is a little hard because (a pattern?) it's a problem with a lot of claims either way in the literature that fail due to subtleties. still im gonna try, two screenshots attached, but i also got opus 5 to write every detail down in a certainly far too long 100+ page pdf

the first page, or at most the first two, have enough to replicate i think



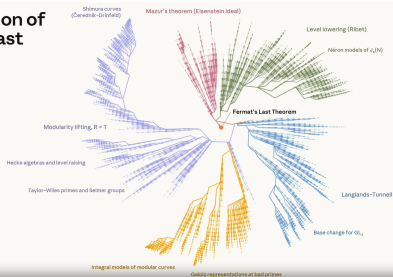
levent @__alpage__ · Aug 23

here's the pdf: [alpo.ge/s6.pdf](#)



We are sharing the first complete computer-checked proof of Fermat's Last Theorem. Claude worked largely autonomously over 11 days to write the proof in the Lean programming language. Below, we describe how the formalization was done and share some thoughts about what this work could mean for research mathematics.

Formalization of Fermat's Last Theorem



FLT: Anthropic has beaten me to it

Posted on September 4, 2026 by [xenaproject](#)

I guess technically it was revealed to the world by [a coffee shop in Islington](#) on Insta, but an hour later it was [officially announced by Anthropic](#): one of their internal models, using the [prove2.me](#) platform, has formalized a complete proof of Fermat's Last Theorem (FLT) in Lean. This is the final theorem to be formalized in Freek Wiedijk's famous [list of 100 formalization challenges](#) and thus wraps up this 20-year-old benchmark. Congratulations to Anthropic!

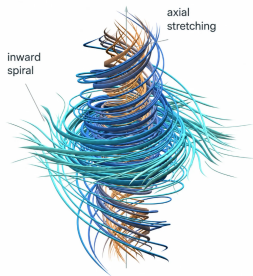


September 8, 2026 Research Publication

On the Navier–Stokes Millennium Prize Problem

[Read the paper](#)

[Link to Lean formalized proof](#) >



On Tuesday, September 1, we heard rumors that two Millennium Prize problems had been resolved. Inspired by these rumors and by the step change in performance of our internal model, we launched an effort to evaluate it on all open Millennium Prize problems and a few other high-impact problems.

We used a system of coordinating agents powered by our internal model. The agents had access to tools such as the ability to read from a cached version of the internet and the ability to run code. Agents were subdivided into groups with the ability to communicate within the group. The groups varied in size, and the group that produced the Navier–Stokes resolution involved on the order of 10,000 concurrent agents. At all times we maintained the same strict safeguards that we apply to all our frontier model evaluations, including monitoring and isolation.

...

The agents arrived at their resolution on Saturday, September 5, about 88 hours after the first agents were launched. Lean formalization and verification took an additional 17 hours via GPT-6 Astra.

Across all attempted problems, the agents sent 4.9 million messages and used about 300 billion output tokens. In the process of resolving the Navier–Stokes problem, the agents sent 2.7 million messages and used approximately 130 billion output tokens.

The future?



The future [of (auto)formalization] is already here — it's just not very evenly distributed.

A Dataset of Modern Formalized Theorem Statements

The Project

This project creates a public dataset of hundreds of formalized statements of recent theorems from top journals, such as the *Annals of Mathematics*. In doing so, it will address a key lack of formalization at the frontier of mathematical research. Through dedicated expert formalization, this project will significantly expand formalized mathematics libraries and provide clear targets for AI systems on a wide range of tasks, including auto-formalizing proofs and assisting humans in proof formalization.

A team of **Lean** experts formalized 50 theorem statements — the “easiest 50” — from the roughly 200 papers published in the *Annals of Mathematics* since 2020.



Overview

A challenging aspect of computer formalization is formalizing definitions that are (i) mathematically correct (something that is not automatically verifiable and must be checked by human experts), (ii) optimized for usability and (iii) at the appropriate level of generality. The AxiOM "Building the mathematical library of the future" will focus on formalizing definitions in Lean across a wide range of mathematical subject areas in the hope that it will be possible to correctly formalize the statements of recent theorems from the research literature. Such work is essential for making formal verification tools more relevant to mathematics researchers.



MATHEMATICS

OPENAI

OpenAI Research Catalog

372 result families in 719 manuscripts

October 6, 2026

An overview of the manuscript collection. Results are grouped by subject; entry numbers follow the catalog order and do not indicate a ranking. PDF links follow each summary.

Contents

Number theory	2
Algebraic and complex geometry	5
Real and complex analysis	9
Convex and metric geometry	10
Theoretical computer science	12
Dynamical systems and ergodic theory	16
Combinatorics	17
Algebra	21
Probability and statistical mechanics	23
Mathematical logic	26
Group theory	27
Mathematical physics	28
Operator algebras	31
Topology	33
Functional analysis	35
Differential geometry	36
Partial differential equations	39

October 6, 2026 Research Publication

Sharing AI progress in mathematics

[View on GitHub ↗](#)

Of the 26 papers in topology, only three come with any **Lean** formalizations, and, in one of the three, the formalization does not cover the main theorem.



3

Predications about the future

We may change the implicit foundation of mathematics



Aside: modern proof assistants often use a newer formal system — **dependent type theory** — in place of traditional Zermelo-Fraenkel set theory and first order logic.

Reintroduction to Proofs

An introduction to proofs in dependent type theory.



$A\ B\ C : \text{Type} \vdash (A \times B \rightarrow C) = (A \rightarrow B \rightarrow C)$
 $P\ Q\ R : \text{Prop} \vdash (P \wedge Q \rightarrow R) \leftrightarrow (P \rightarrow Q \rightarrow R)$

This game introduces mathematical proof writing in a formal vocabulary that is designed to make it easier to be fully precise about every step in a proof, as is required to convince a computer that a logical argument is sound.

The language of dependent type theory can be used in parallel to state and prove mathematical theorems as well as to specify and define examples of mathematical objects.

Prerequisites

Worlds	17
Levels	155
Language	en

In Fall 2025, I taught an experimental introduction to proofs in informal dependent type theory using **Lean**.

The result is the **Reintroduction to Proofs** game hosted on the **The Lean Game Server**:

adam.math.hhu.de

In my view, informal dependent type theory better reflects mathematical practice: the formal definitions of fundamental concepts more closely correspond to the way mathematicians think about them.

Norm: AI-generated proofs must be formalized



A proposed norm for original mathematics produced by an AI system:

Any artificially generated mathematical text will **not be considered as a proof** unless:

- It has been communicated in both a natural language text paired with a computer formalization of all definitions, theorems, and proofs.
- The formalization has been accepted by the proof assistant and human expert referees have vetted both the formalization and the paired text.

An unformalized AI-generated mathematical argument is
a “potential proof” or “purported proof.”

Open question: Can autoformalization support libraries of formalized mathematics?

High impact human-generated mathematics will be “post-rigorous”



Correctness, Artificial Intelligence, and the
Epistemic Value of Mathematical Proof

James Owen Weatherall

*Department of Logic and Philosophy of Science
University of California, Irvine*

Jesse Wolfson

*Department of Mathematics
University of California, Irvine*

“That a mathematical proof be correct
is neither sufficient nor necessary for it to have
epistemic value.”

— James Owen Weatherall and Jesse Wolfson

“There’s more to mathematics than
rigour and proofs.”

— Terry Tao

The history of every major galactic civilization tends to pass through three distinct and recognizable phases, those of Survival, Inquiry and Sophistication, otherwise known as the How, Why, and Where phases. For instance, the first phase is characterized by the question ‘How can we eat?’, the second by the question ‘Why do we eat?’ and the third by the question, ‘Where shall we have lunch?’ (Douglas Adams, “[The Hitchhiker’s Guide to the Galaxy](#)”)

Dream: with details “left to the formalization” rather than to the reader, perhaps papers can be shorter?

Proof aesthetics still matter



“The mathematician’s patterns, like the painter’s or the poet’s must be beautiful; the ideas like the colours or the words, must fit together in a harmonious way. Beauty is the first test: there is no permanent place in the world for ugly mathematics.”

– G.H. Hardy, *A Mathematician's Apology*

I would argue this holds for formal proofs as well as pen-and-paper proofs.

(Aesthetic considerations about formal proofs also argue strongly
for a diverse ecosystem of multiple proof assistants
implementing a variety of formal systems
...but that is a discussion for another time.)

Tusen takk!